IST608 CA – Advanced Database Systems

Second Semester Examination 2025/2026 — Detailed Model Answers

University of Buea · Department of Computer Science · Faculty of Science

QUESTION 1 (20 Marks) — Database Recovery

Part A (8 Marks) — Undo and Redo in Crash Recovery

i. Operational Differences: Roll-back (Undo) vs Roll-forward (Redo)

When a system crashes and restarts, the Recovery Manager reads the **write-ahead log (WAL)** and classifies every transaction into one of two lists:

- **Undo List** — transactions that were **active (not yet committed)** at the time of the crash.
- **Redo List** — transactions that had **committed** but whose updates may not yet have been flushed from the buffer pool to disk.

Roll-back / UNDO

Aspect	Description
Purpose	Remove the partial, uncommitted effects of a failed transaction so the database is returned to a consistent state before that transaction began.
Direction	Processes the log **backwards** (from the most recent log record toward the oldest) to reverse updates in the correct sequence.
Applied to	Transactions in the **Undo List** (those with a `START` record but **no** `COMMIT` record in the log).
Log record used	Uses the `BFIM` (Before Image) — the old value stored in each `<T, X, old_val, new_val>` log record — to restore each modified data item X to its old value.
Writes to log	Writes **compensation log records (CLRs)** documenting each undo step, so if the system crashes again during undo, it can resume without re-doing already-undone work.
Outcome	The transaction's changes are entirely erased; the transaction is rolled back to its implicit `START`.

Roll-forward / REDO

Aspect	Description

Aspect	Description
Purpose	Re-apply the committed changes of a transaction whose updates may have been lost because the dirty buffer pages were not written to disk before the crash.
Direction	Processes the log **forwards** (from the oldest relevant record to the most recent) to re-apply updates in the original order.
Applied to	Transactions in the **Redo List** (those with both a `START` and a `COMMIT` record in the log).
Log record used	Uses the `AFIM` (After Image) — the new value stored in each `<T, X, old_val, new_val>` log record — to write the committed value of X to disk.
Idempotent	Applying the same redo operation twice is safe; the result is the same as applying it once, because the log contains the exact final value.
Outcome	The database reflects all committed changes, even those that had not yet been flushed to disk at crash time.

ii. Step-by-Step Execution Trace

Consider the following scenario with two transactions sharing a data item X:

```
Time →
T1:  START  →  Write(X, 50)  →  COMMIT       (committed just before crash)
T2:  START  →  Write(X, 80)             →  CRASH  (active at crash time)
```

Log on disk at the time of crash:

```
LSN  Record
1    <T1, START>
2    <T1, X, old=10, new=50>
3    <T1, COMMIT>
4    <T2, START>
5    <T2, X, old=50, new=80>
     [CRASH — buffer not flushed; disk still has X = 10 or X = 50 or X = 80
depending on buffer eviction]
```

Phase 1 — Analysis Pass (Recovery Manager scans full log forward)

- T1 has START + COMMIT → placed in **Redo List**.
- T2 has START but no COMMIT → placed in **Undo List**.

Phase 2 — Redo Pass (scan log forward from LSN 1)

```
LSN 2: T1 wrote X = 50 → RE-APPLY: set X = 50 on disk
LSN 3: T1 COMMIT → confirmed; T1 is durably committed
```

```
LSN 5: T2 wrote X = 80 → RE-APPLY: set X = 80 on disk
       (Redo even uncommitted transactions — correctness ensured by the Undo pass)
```

After Redo pass: X = 80 on disk.

Phase 3 — Undo Pass (scan log backward from LSN 5)

```
LSN 5: T2 wrote X = 80 using old=50 → UNDO: set X = 50 on disk
       Write CLR: <T2, X, undone-to=50>
LSN 4: T2 START → write <T2, ABORT> to log
```

After Undo pass: X = 50 on disk (reflects only T1's committed update).

Final consistent state: X = 50 — T1's committed change is preserved; T2's partial change is erased.

Part B (6 Marks) — Checkpointing

i. Sequence of Actions During a Standard Checkpoint

Checkpointing is the mechanism by which the DBMS periodically creates a known consistent point in the log, enabling faster recovery. The standard (sharp/simple) checkpoint proceeds as follows:

Step 1 — Suspend new transactions temporarily The DBMS momentarily stops accepting new write operations to ensure a clean snapshot.

Step 2 — Flush all dirty buffer pages to disk All modified (dirty) pages currently in the buffer pool are written to their permanent locations on disk. This ensures that any changes logged up to this point are actually reflected on disk.

Step 3 — Write the checkpoint record to the log A special log record `<CHECKPOINT, {T_active}>` is written. It lists every transaction currently active (started but not yet committed). Example:

```
<CHECKPOINT, {T3, T5}>
```

Step 4 — Flush the log buffer to disk The log up to and including the checkpoint record is forced to stable (disk) storage. This ensures the checkpoint record itself survives a subsequent crash.

Step 5 — Resume normal operations Transactions are allowed to proceed again.

---

ii. How Checkpointing Limits Log Scanning

Without checkpointing, after a crash the Recovery Manager would need to scan the **entire log from the very beginning** to determine which transactions to redo or undo — a log that can grow to many gigabytes over days or months.

With checkpointing, recovery only needs to scan back to the **most recent checkpoint record** because:

1. **All transactions that committed before the checkpoint** had their changes already flushed to disk in Step 2. Their updates are durable without any redo action.
2. **The checkpoint record explicitly lists** which transactions were active at checkpoint time. These are the only transactions that might need undo.
3. **Any transaction that started after the checkpoint** can be found by scanning forward from the checkpoint record — a much shorter section of the log.

Diagram:

```
Log:
|← ancient history ——|← CHECKPOINT —|← recent activity —|CRASH
                              ↑
                   Recovery starts here, not at beginning

Zone before CHECKPOINT: Guaranteed on disk → SKIP (no redo needed)
Zone after CHECKPOINT:  Must be analyzed  → REDO committed; UNDO active
```

Practical impact: If the DBMS checkpoints every 5 minutes, the worst-case recovery scan covers only 5 minutes of log records, regardless of how long the database has been running.

Part C (6 Marks) — Recovery in Distributed / Multidatabase Transactions

i. Complexities of Recovery in Multidatabase Transactions

A distributed transaction spans multiple independent database sites (nodes), each with its own local Recovery Manager and log. This introduces several critical complexities:

1. No Shared Global Log Each site maintains only its own local log. There is no single global log that captures the full execution order of a distributed transaction. The Recovery Manager at Site A has no visibility into what Site B logged.

2. Partial Commit / Partial Failure A transaction may successfully commit at Sites A and B, but crash before committing at Site C. This leaves the database in a **globally inconsistent state** even though each site is locally consistent. Without a coordination protocol, Site C would undo what Sites A and B have committed.

3. Independent Site Failures Any participant (or the coordinator itself) can fail independently and at any time — before sending a vote, after sending a vote, or during commit. Each combination creates a different recovery scenario requiring different handling.

4. Network Partitions A site may be alive but unreachable due to network failure. The coordinator cannot distinguish between a crashed participant and a slow/partitioned one, making it impossible to proceed without risking inconsistency.

5. Blocking Problem If the coordinator fails after participants have voted YES and entered the "uncertain" state, participants are **blocked** — they cannot commit (the coordinator may have decided abort) nor abort (the coordinator may have decided commit). They must wait until the coordinator recovers.

6. Cascading Redo/Undo Requirements Compensating a distributed transaction requires coordinated undo across all sites. If compensation itself fails mid-way (e.g., Site B goes down while Site A is being undone), the system may be left in a state that is difficult to reconcile.

---

ii. The Two-Phase Commit (2PC) Protocol

2PC is the standard protocol for ensuring **atomic commit** in distributed transactions — either all sites commit or all sites abort, with no partial outcomes.

Participants: One **Coordinator** (C) and one or more **Participants** (P1, P2, ..., Pn).

Phase 1 — Voting Phase (Prepare Phase)

```
Coordinator:  PREPARE  →  [broadcast to all participants]
                          ┌──────────────┐
                          │  P1          │ → writes "prepared" to local log
                          │  P2          │ → writes "prepared" to local log
                          │  ...         │
                          └──────────────┘
                                ↓
                          VOTE_YES / VOTE_NO  →  [send back to coordinator]
```

1. The coordinator sends a `PREPARE` message to all participants.
2. Each participant:
   - Checks if it can commit (all local constraints satisfied, no conflicts).
   - If **YES**: flushes its local log to disk, writes a `PREPARED` log record (force to disk), sends `VOTE_YES`.
   - If **NO**: writes `ABORT` to local log, sends `VOTE_NO`, and locally aborts immediately.

Phase 2 — Decision Phase (Commit/Abort Phase)

```
If ALL votes = YES:
    Coordinator writes COMMIT to log, broadcasts COMMIT to all participants.
    Each participant: commits locally, writes COMMIT, releases locks, sends ACK.

If ANY vote = NO (or timeout):
    Coordinator writes ABORT to log, broadcasts ABORT to all participants.
    Each participant: undoes local changes, writes ABORT, releases locks, sends
ACK.

Coordinator: on receiving all ACKs → writes END record (transaction complete).
```

Recovery from Failures

Failure Scenario	Recovery Action
Participant fails before voting	Coordinator times out → decides ABORT → broadcasts ABORT. On recovery, participant checks its log: if no PREPARED record → locally abort (no harm done).
Participant fails after voting YES	On recovery, participant checks log: PREPARED record exists but no COMMIT/ABORT. It **contacts the coordinator** to ask the outcome, then applies it. (It must not unilaterally abort — it already voted YES.)
Coordinator fails before sending PREPARE	Participants have no PREPARED record → all locally abort.
Coordinator fails after some participants voted YES	Participants with PREPARED records are **blocked** — they cannot proceed without the coordinator's decision. On coordinator recovery, it reads its log: if COMMIT record exists → re-broadcasts COMMIT; if no decision → decides ABORT.
Coordinator fails after writing COMMIT to log	On recovery, coordinator re-broadcasts COMMIT to any participants that did not yet acknowledge. The COMMIT record in the coordinator's log is the durable decision point.

Key guarantee: The atomic durability of the decision is ensured because the coordinator **writes the COMMIT or ABORT record to its log and forces it to disk** before broadcasting. This log record is the single authoritative source of truth, surviving all subsequent failures.

QUESTION 2 (28 Marks) — Concurrency Control

Part A (8 Marks) — Schedules and Recoverability

i. Definition of a Schedule (History)

A **schedule** (also called a *history*) is a formal ordered sequence of the operations from a set of concurrently executing transactions, where:

- The **relative order** of operations **within each individual transaction** is preserved (as specified by the transaction's program).
- Operations from **different transactions may be interleaved** in any order (reflecting the actual interleaved execution by the DBMS scheduler).

Formally: Given transactions T_1, T_2, …, T_n, a schedule S is a total ordering of all operations from these transactions such that for each T_i, the operations of T_i in S appear in the same order as they do in T_i's program.

Operations included in a schedule: `read(X)`, `write(X)`, `commit`, `abort` (and sometimes `begin`).

Example:

```
T1: r(X), w(X), commit
T2: r(X), w(X), commit

Possible Schedule S:
r₁(X), r₂(X), w₁(X), commit₁, w₂(X), commit₂
```

---

ii. Recoverable, Cascadeless (Avoids Cascading Aborts), and Strict Schedules

These form a **containment hierarchy** of increasingly restrictive correctness conditions:

```
All Schedules
  └── Recoverable Schedules
        └── Cascadeless (ACA) Schedules
              └── Strict Schedules
```

---

1. Recoverable Schedule

Formal condition: A schedule S is recoverable if and only if — for every pair of transactions T_i and T_j such that T_j reads a value that was last written by T_i — **T_i commits before T_j commits**.

Rule: No transaction commits until all transactions whose values it has read have already committed.

Why needed: If T_j reads a dirty value from T_i and commits before T_i does, and T_i later aborts, then T_j committed based on data that never officially existed — an unrecoverable situation (cannot roll back T_j).

Example — Non-Recoverable (BAD):

```
r₁(X), w₁(X), r₂(X), commit₂, abort₁
```

T_2 read X written by T_1 (dirty read), committed, then T_1 aborted. T_2 can never be undone — inconsistent.

Example — Recoverable:

```
r₁(X), w₁(X), r₂(X), commit₁, commit₂
```

T_1 commits before T_2 commits — safe.

---

2. Cascadeless Schedule (Avoids Cascading Aborts — ACA)

Formal condition: A schedule S is cascadeless if and only if — for every pair of transactions T_i and T_j such that T_j reads a value that was last written by T_i — **T_i commits before T_j reads that value**.

Rule: Transactions may only read values from **already-committed** transactions.

Why stricter than Recoverable: In a merely recoverable schedule, T_2 can read dirty data from T_1 as long as it delays its own commit until T_1 commits. But if T_1 aborts, T_2 must also be aborted — which may force T_3 (which read T_2's dirty data) to abort as well — causing a **cascade of aborts** that is expensive and disrupts many concurrent users.

Cascadeless schedules prevent dirty reads entirely, so no cascade can occur.

Example — Cascadeless:

```
r₁(X), w₁(X), commit₁, r₂(X), w₂(X), commit₂
```

T_2 does not read X until after T_1 has committed.

---

3. Strict Schedule

Formal condition: A schedule S is strict if and only if — for every pair of transactions T_i and T_j such that T_j either **reads or writes** a value that was last written by T_i — **T_i commits (or aborts) before T_j reads or writes that value**.

Rule: No transaction may read *or write* a data item that has been written (but not yet committed or aborted) by another transaction.

Why strictest: In a cascadeless schedule, a transaction T_j may still overwrite a value written by an active T_i, creating a **lost update** problem during undo. In a strict schedule, the simple undo mechanism (restore the Before Image) is always correct, because uncommitted writes are never overwritten by other transactions.

Practical importance: Strict schedules are the theoretical foundation for **Strict Two-Phase Locking (Strict 2PL)**, the most common industrial locking protocol, which holds all **exclusive (write) locks until commit**.

Summary Table:

Property	Dirty Read by T_j	Dirty Write by T_j	Cascade risk	Undo safety
Recoverable	Allowed (commit-ordered)	Allowed	Yes (costly)	Complex
Cascadeless (ACA)	Forbidden	Allowed	No	Moderate
Strict	Forbidden	Forbidden	No	Simple (BFI always correct)

Part B (6 Marks) — Recoverability Analysis of SchedA

Schedule SchedA: $r_1(X);\ w_1(X);\ r_2(X);\ w_2(X);\ a_1$

Where: r = read, w = write, a = abort, subscript = Transaction ID.

i. Classification of SchedA

Step 1 — Identify data dependencies:

- $w_1(X)$ at position 2 → $r_2(X)$ at position 3: **T_2 reads a value written by T_1** (T_1 has not yet committed — this is a **dirty read**).

Step 2 — Apply the three definitions:

Is SchedA Recoverable? For SchedA to be recoverable, T_1 must commit before T_2 commits. However, T_1 **aborts** (a_1) at the end instead of committing. T_2 read T_1's dirty value. Since T_1 never commits (it aborts), and T_2 has implicitly used that dirty data, this schedule is problematic. However, note that T_2 does not explicitly commit in this schedule — $w_2(X)$ is the last T_2 operation shown. If we assume T_2 has not yet committed before T_1's abort is processed, then recovery (aborting T_2 as well) is still possible. But the schedule as given **requires T_2 to also be aborted** because T_1 aborted. Whether it is recoverable depends on whether T_2 commits after a_1 or not.

Since a_1 appears last and T_2 has no commit shown: The schedule as written does not reach a state where T_2 has committed while T_1 has not — so it is **recoverable in principle**, but it **triggers a cascading abort**.

Is SchedA Cascadeless (ACA)? No. T_2 reads X at position 3 ($r_2(X)$) **before T_1 commits**. T_1 has only written X at position 2 ($w_1(X)$) — it is still active. This is a dirty read. A cascadeless schedule requires that T_2 only read values from already-committed transactions. Since T_1 has not committed when T_2 reads X, SchedA **violates the ACA condition**.

Is SchedA Strict? No. T_2 reads X ($r_2(X)$) and writes X ($w_2(X)$) while T_1 has an uncommitted write on X. A strict schedule requires that T_2 may not read or write X until T_1 commits or aborts. This condition is violated at both positions 3 and 4.

Conclusion: SchedA is classified as **Recoverable but neither Cascadeless nor Strict**. (It may technically be recoverable since T_2 does not commit before a_1 — but it is at the lowest/weakest tier of the recoverability hierarchy.)

---

ii. Dynamic Phenomenon When T_1 Issues a_1 — Cascading Abort

When T_1 issues the abort command (a_1), the following **cascade of aborts** is triggered:

What happens:

1. **T_1 aborts** — the Recovery Manager undoes T_1's write by restoring the before-image of X (its original value before $w_1(X)$).
2. **T_2 read a dirty value** — T_2 read the value of X at position 3 ($r_2(X)$), which was the value written by T_1 ($w_1(X)$). Since T_1 has now aborted, that value never officially existed.
3. **T_2 must also be aborted** — because T_2's read is now meaningless (based on uncommitted, rolled-back data), and T_2's write ($w_2(X)$) was based on that dirty read. Allowing T_2 to continue would produce a logically inconsistent result.

4. **T_2 is aborted** — $w_2(X)$ is undone; X is restored to its value before $w_2(X)$.

This is a Cascading Abort: The abort of T_1 forces the abort of T_2 simply because T_2 read T_1's uncommitted data.

Impact on System Performance:

Impact	Description
Wasted work	All operations of T_2 are rolled back, even though T_2 itself may have had no logical error. CPU cycles, I/O, and lock acquisitions are all wasted.
Increased lock holding time	T_2 must be aborted and its locks released; during this time, other transactions waiting for those locks are further delayed.
Potential chain reaction	If a T_3 had read T_2's dirty write ($w_2(X)$), T_3 must also be aborted — potentially cascading to T_4, T_5, and so on.
Reduced throughput	In high-concurrency workloads, cascading aborts can cause a domino effect that dramatically reduces the number of transactions that successfully commit per second.
Recovery overhead	Each aborted transaction requires log processing, before-image restoration, and lock release — multiplying the recovery cost proportionally to the cascade depth.

Prevention: Use **Strict Two-Phase Locking (Strict 2PL)** or **Snapshot Isolation**, both of which prevent dirty reads entirely, eliminating the possibility of cascading aborts.

Part C (6 Marks) — Isolation Levels and SQL Anomalies

i. The Three Operational Anomalies

1. Dirty Read

A **dirty read** occurs when a transaction T_2 reads a data item X that has been modified by another transaction T_1, but T_1 **has not yet committed**. If T_1 subsequently aborts, T_2 has read a value that never officially existed in the database.

Example:

```
T1: w(balance = 500)      -- T1 decrements balance
T2:     r(balance)        -- T2 reads 500 (dirty!)
T1:         ABORT         -- T1 rolls back; balance reverts to 1000
T2 now has the wrong value 500 — it read data that doesn't exist
```

---

2. Non-Repeatable Read

A **non-repeatable read** occurs when a transaction T_1 reads a data item X twice within the same transaction and gets **different values** each time, because another transaction T_2 modified and committed X between T_1's two reads.

Example:

```
T1: r(salary)         -- reads 3000
T2:     w(salary=4000), COMMIT
T1: r(salary)         -- reads 4000 (different from first read!)
```

T_1 cannot reproduce its own earlier result within the same transaction — the data has changed beneath it.

---

3. Phantom Read

A **phantom read** occurs when a transaction T_1 executes a **range query** (e.g., `SELECT WHERE age > 30`) twice and gets a **different set of rows** each time, because another transaction T_2 has inserted or deleted rows that satisfy the query's predicate between the two executions.

Example:

```
T1: SELECT * FROM Employees WHERE dept='CS'  -- returns 10 rows
T2:     INSERT INTO Employees(dept='CS', ...)  -- adds 1 new CS employee, COMMIT
T1: SELECT * FROM Employees WHERE dept='CS'  -- returns 11 rows (phantom row
appeared!)
```

The "phantom" is the newly inserted row that was invisible in the first query but appears in the second.

---

ii. Mapping Anomalies to SQL Isolation Levels

The SQL standard defines four isolation levels, each preventing a specific set of anomalies:

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read	Typical Implementation
READ UNCOMMITTED	✅ Permitted	✅ Permitted	✅ Permitted	No read locks acquired
READ COMMITTED	❌ Prevented	✅ Permitted	✅ Permitted	Short-duration read locks (released immediately after read)
REPEATABLE READ	❌ Prevented	❌ Prevented	✅ Permitted	Long-duration read locks (held until commit), but no range locks
SERIALIZABLE	❌ Prevented	❌ Prevented	❌ Prevented	Long-duration read + range/predicate locks; or MVCC with full serialization

Explanation of each level:

READ UNCOMMITTED: The lowest isolation level. Transactions read directly from the buffer/disk without acquiring any read locks. All three anomalies are possible. Rarely used except for approximate, statistical queries where absolute accuracy is not required (e.g., read a rough count of rows).

READ COMMITTED: Prevents dirty reads by acquiring a **shared lock** on X when reading it, but releasing that lock immediately after the read completes. Because the lock is released before the transaction commits, another transaction can modify X before T_1 reads it again — causing non-repeatable reads.

REPEATABLE READ: Prevents dirty reads and non-repeatable reads by holding shared locks on **every individual data item read** until the transaction commits. However, it does not lock gaps or ranges, so new rows satisfying a predicate can be inserted — causing phantom reads.

SERIALIZABLE: The highest isolation level. Prevents all three anomalies. Implemented either via:

- **Predicate (range) locking** — locks entire key ranges so no insertions can satisfy the predicate.
- **MVCC (Multiversion Concurrency Control)** with serialization conflict detection (used in PostgreSQL, Oracle).

Performance vs. Isolation Trade-off: Lower isolation levels allow more concurrency (fewer/shorter locks) at the cost of data anomalies. Higher levels guarantee correctness but reduce throughput due to increased lock contention.

Part D (8 Marks) — Serializability

i. Serializability vs. Serial vs. Nonserial Schedules

Serial Schedule

A schedule is **serial** if the transactions execute **one at a time, in sequence**, with no interleaving — each transaction runs to completion before the next begins.

```
Serial Schedule Example (T₁ then T₂):
r₁(X), w₁(X), commit₁, r₂(X), w₂(X), commit₂
```

Serial schedules are **always correct** (consistent) because transactions cannot interfere with each other. However, they are highly inefficient — each transaction must wait for the entire previous transaction to finish, even if it needs a completely different resource.

Nonserial Schedule

A schedule is **nonserial** if the operations of multiple concurrent transactions are **interleaved** — one transaction's operations are mixed with another's, rather than executed in strict sequence.

```
Nonserial Schedule Example:
r₁(X), r₂(X), w₁(X), commit₁, w₂(X), commit₂
```

Nonserial schedules can significantly improve system **throughput and resource utilization** (CPU, I/O, memory can be used in parallel). However, they may introduce anomalies if not carefully managed.

Serializable Schedule

A nonserial schedule S is **serializable** if it produces the **same final database state and the same output** as some serial schedule over the same set of transactions.

More formally: S is serializable if it is **equivalent** to some serial schedule, where equivalence is typically defined as:

- **Conflict Serializability:** S is equivalent to a serial schedule if all **conflicting operations** (pairs of operations from different transactions on the same data item where at least one is a write) appear in the same relative order as in some serial schedule. Tested via the **Precedence Graph (Serialization Graph)**: if the graph is acyclic, S is conflict-serializable.
- **View Serializability:** A more general (but harder to check) equivalence — S is view-serializable if the set of reads-from relationships and final writes match some serial schedule.

Key distinction:

```
Serial ⊂ Serializable ⊂ Nonserial

All serial schedules are trivially serializable.
Not all nonserial schedules are serializable.
Serializability identifies which nonserial schedules are safe.
```

---

ii. Why Enforcing Strictly Serial Schedules is Impractical in High-Performance Production Environments

Enforcing a strictly serial execution model — where only one transaction runs at a time — would be catastrophic for production database performance for the following reasons:

1. Massive Throughput Reduction Modern production databases handle thousands to millions of transactions per second (e.g., banking systems, e-commerce). Serial execution means each transaction waits for all preceding ones to finish. If each transaction takes 10ms, a serial system can do only 100 TPS. A concurrent system with 50 concurrent threads can do ~5,000 TPS — a 50× improvement.

2. Wasted I/O Wait Time The majority of transaction time is spent waiting for **disk I/O** (reading/writing pages). In serial execution, the CPU and other transactions sit idle during every I/O wait. With interleaving, while T_1 waits for a disk read, T_2 can execute CPU-bound operations — dramatically improving hardware utilization.

3. Long Transactions Block Everything In a serial model, a single long-running transaction (e.g., a report query scanning millions of rows for 5 minutes) blocks all other transactions from executing for 5 minutes. In a concurrent model, short transactions can execute alongside the long one, touching different data.

4. Deadlock Elimination (False Argument) Serial execution avoids deadlocks — but the cost is prohibitive. Deadlock detection and resolution in a concurrent system has negligible overhead compared to the performance gains of concurrency.

5. Multi-Core and Distributed Systems Modern servers have 32, 64, or 128 CPU cores, and distributed databases span hundreds of nodes across data centers. Serial execution utilizes only one core and one node at a time — an egregious waste of hardware investment.

6. User Experience In a serial database, a user submitting a form waits behind every other user's transaction globally. Interactive applications (web, mobile) require sub-second response times that are simply impossible under serial execution at scale.

The Solution — Concurrency Control: Production systems use **concurrency control protocols** (Two-Phase Locking, MVCC, Optimistic Concurrency Control) to guarantee **serializability** — the correctness of serial schedules — while preserving the performance benefits of concurrent, interleaved execution. This is the fundamental bargain of the concurrency control subsystem: achieve the *correctness guarantee* of serial execution without its *performance penalty*.

End of Model Answers — IST608 CA Advanced Database Systems